Transfer learning, and going deep

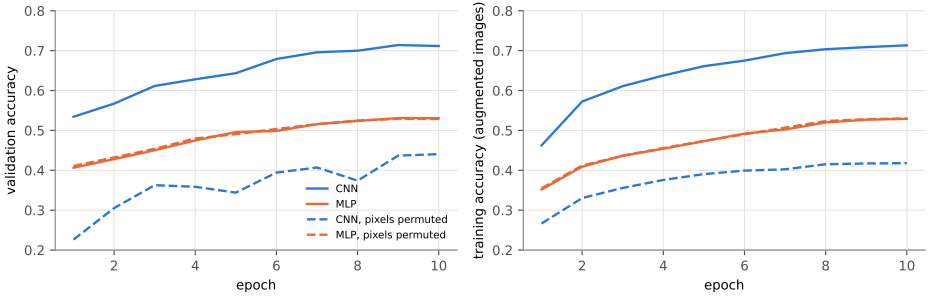
AI Methods for Science – Part I – Week 3, Thursday

Yohai Bar-Sinai

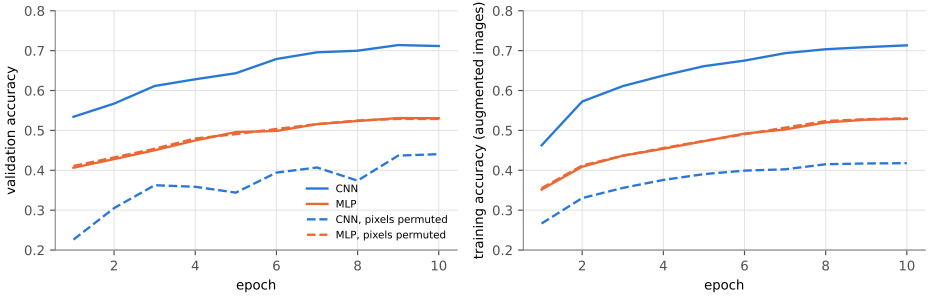
October 29, 2026

Monday's lab, from our runs

E16: the CNN, the MLP, and the permutation

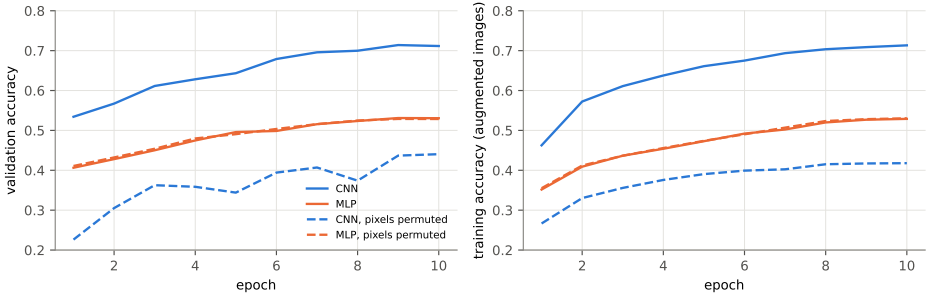


E16: the CNN, the MLP, and the permutation



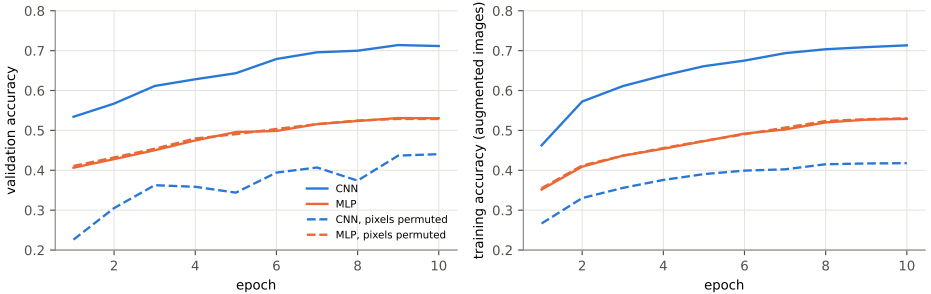
- 10 epochs, the same recipe: CNN 71.2 with $95{,}000$ parameters; MLP 53.1 with 1.7M. $18\times$ fewer parameters, $+18$ points.

E16: the CNN, the MLP, and the permutation



- 10 epochs, the same recipe: CNN 71.2 with $95{,}000$ parameters; MLP 53.1 with 1.7M. $18\times$ fewer parameters, $+18$ points.
- The MLP does not notice the permutation, $53.1 \to 52.9$: it never used the geometry. The CNN drops to 44: the prior is now wrong about the data, yet still above the MLP (some true neighbours land together; pooling aggregates).

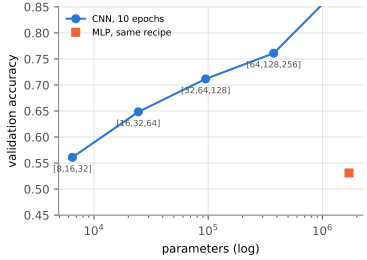
E16: the CNN, the MLP, and the permutation

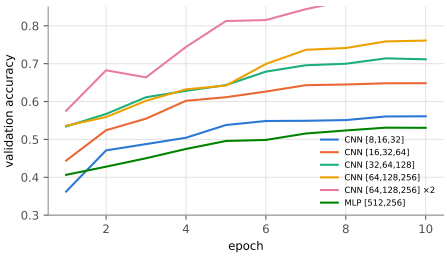


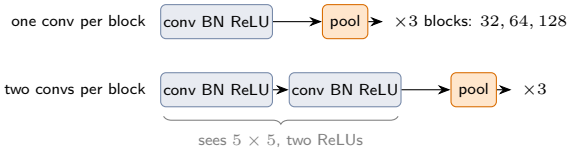
- 10 epochs, the same recipe: CNN 71.2 with $95{,}000$ parameters; MLP 53.1 with 1.7M. $18\times$ fewer parameters, $+18$ points.
- The MLP does not notice the permutation, $53.1 \to 52.9$: it never used the geometry. The CNN drops to 44: the prior is now wrong about the data, yet still above the MLP (some true neighbours land together; pooling aggregates).

What the CNN knows is which pixels are neighbours: worth 27 points here, and not learnable from $45{,}000$ images.

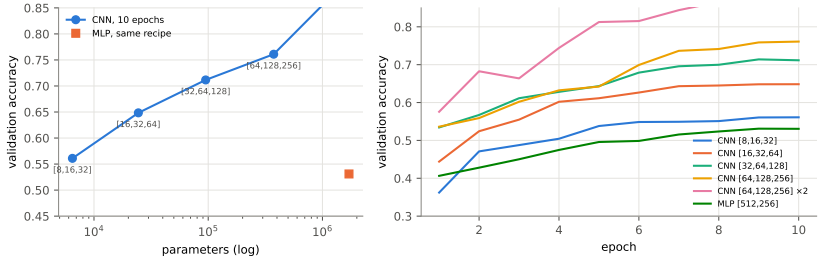
E17: width and depth

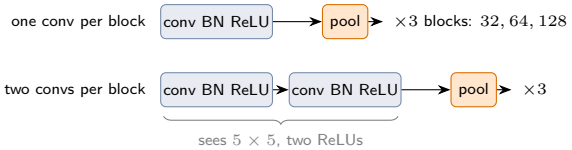






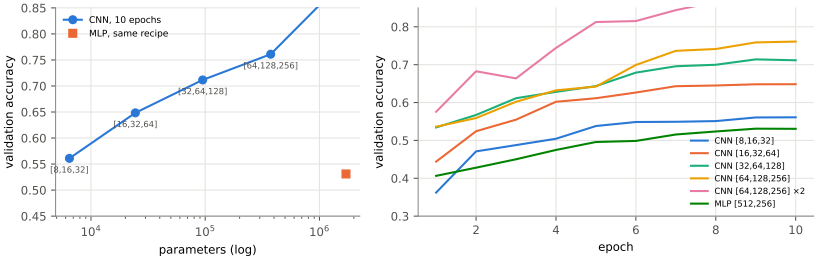
E17: width and depth

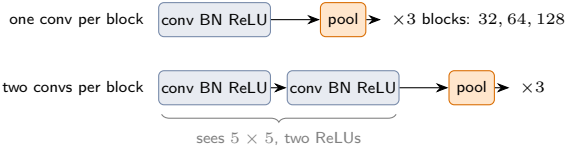




- Width, each doubling: $4\times$ the parameters, ≈ 5 points.

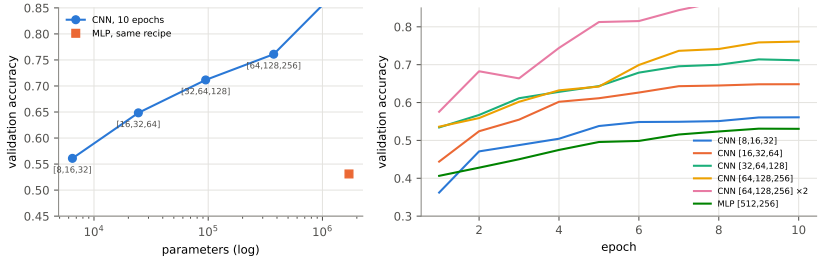
E17: width and depth

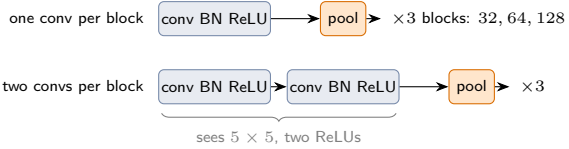




- Width, each doubling: $4\times$ the parameters, ≈ 5 points.
- Depth, a second conv in each block: $3\times$ the parameters, $+11$ points ($76.1 \to 86.9$, the pink curve).

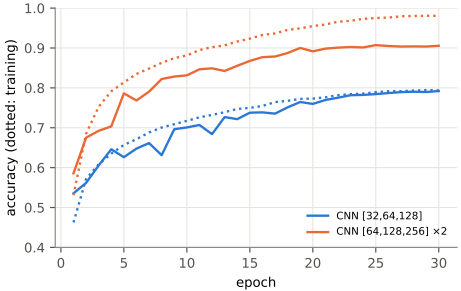
E17: width and depth

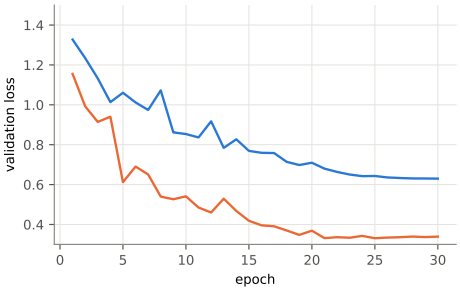




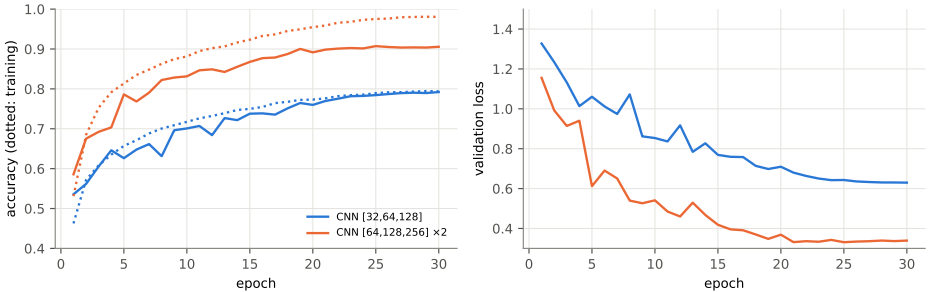
- Width, each doubling: $4\times$ the parameters, ≈ 5 points.
- Depth, a second conv in each block: $3\times$ the parameters, $+11$ points ($76.1 \to 86.9$, the pink curve).
- **A 6k-parameter CNN beats a 1.7M MLP.** The prior is the currency.

E18: the best CNN, 30 epochs, test once



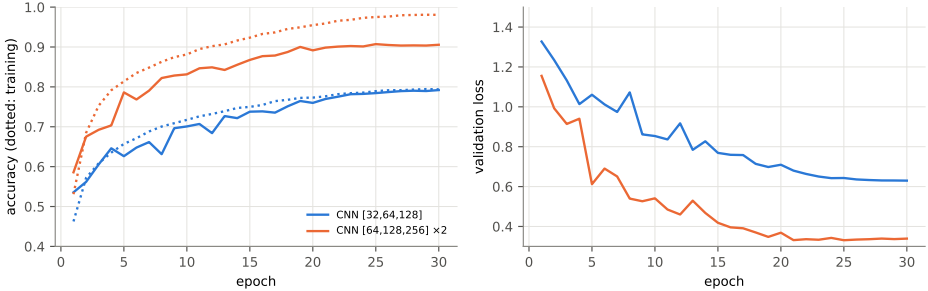


E18: the best CNN, 30 epochs, test once



- The small CNN 79.2; `[64,128,256]` with two convs per block 90.7 on validation, **test** 90.0. From 59 to 90 by changing the ansatz; the loop, the loss and the recipe are week 2's. Training accuracy 98%, validation loss bottoms at epoch 25: with augmentation and cosine the gap is 8 points, not 45 (E4).

E18: the best CNN, 30 epochs, test once



- The small CNN 79.2; `[64,128,256]` with two convs per block 90.7 on validation, **test** 90.0. From 59 to 90 by changing the ansatz; the loop, the loss and the recipe are week 2's. Training accuracy 98%, validation loss bottoms at epoch 25: with augmentation and cosine the gap is 8 points, not 45 (E4).
- The gap to 94% (a human) and 99% (the best networks): deeper with skips, stronger augmentation, longer training, a pretrained backbone. Today: the last two.

Transfer learning

Don't be a hero

In practice: use whatever works best on ImageNet. If you're feeling a bit of a fatigue in thinking about the architectural decisions, you'll be pleased to know that in 90% or more of applications you should not have to worry about these. I like to summarize this point as "*don't be a hero*": Instead of rolling your own architecture for a problem, you should look at whatever architecture currently works best on ImageNet, download a pretrained model and finetune it on your data. You should rarely ever have to train a ConvNet from scratch or design one from scratch. I also made this point at the Deep Learning school.

Andrej Karpathy, CS231n course notes (Stanford), "Convolutional Networks".

Don't be a hero

In practice: use whatever works best on ImageNet. If you're feeling a bit of a fatigue in thinking about the architectural decisions, you'll be pleased to know that in 90% or more of applications you should not have to worry about these. I like to summarize this point as "*don't be a hero*": Instead of rolling your own architecture for a problem, you should look at whatever architecture currently works best on ImageNet, download a pretrained model and finetune it on your data. You should rarely ever have to train a ConvNet from scratch or design one from scratch. I also made this point at the Deep Learning school.

Andrej Karpathy, CS231n course notes (Stanford), "Convolutional Networks".

- For images, the backbone is a solved problem: take the current best on ImageNet, pretrained, and fine-tune. Your effort goes into the data, the labels, the validation protocol, and the head.

Don't be a hero

In practice: use whatever works best on ImageNet. If you're feeling a bit of a fatigue in thinking about the architectural decisions, you'll be pleased to know that in 90% or more of applications you should not have to worry about these. I like to summarize this point as "*don't be a hero*": Instead of rolling your own architecture for a problem, you should look at whatever architecture currently works best on ImageNet, download a pretrained model and finetune it on your data. You should rarely ever have to train a ConvNet from scratch or design one from scratch. I also made this point at the Deep Learning school.

Andrej Karpathy, CS231n course notes (Stanford), "Convolutional Networks".

- For images, the backbone is a solved problem: take the current best on ImageNet, pretrained, and fine-tune. Your effort goes into the data, the labels, the validation protocol, and the head.
- The exception is data that are not photographs: then the workshop's question, what to pretrain on, is yours to answer.

E20 (start now, finish at home): what does a trained network know about classes it never saw?

```
from cnn_lab import train, nearest_mean
train({"classes": [0,1,2,3,4], "epochs": 10,
       "save_weights": "pre.pt"}) # (a) plane..deer
B = [5,6,7,8,9] # dog..truck
for n in (10, 100, 1000): # per class
  nearest_mean("pre.pt", B, n) # (b) no training
  train({"classes": B, "n_per_class": n, "
       init_from": "pre.pt",
         "freeze": True, "lr": 1e-2, "batch_size":
              32,
         "epochs": 100, "augment": False}) # (c)
              head only
  train({"classes": B, "n_per_class": n,
         "init_from": "pre.pt"}) # (d) fine-tune
              all
  train({"classes": B, "n_per_class": n}) # (e)
       from scratch
```

One minute on a GPU, five on a CPU. Collect validation accuracy vs. n, four curves on one axis.

E20 (start now, finish at home): what does a trained network know about classes it never saw?

```
from cnn_lab import train, nearest_mean
train({"classes": [0,1,2,3,4], "epochs": 10,
       "save_weights": "pre.pt"}) # (a) plane..deer
B = [5,6,7,8,9] # dog..truck
for n in (10, 100, 1000): # per class
  nearest_mean("pre.pt", B, n) # (b) no training
  train({"classes": B, "n_per_class": n, "
       init_from": "pre.pt",
        "freeze": True, "lr": 1e-2, "batch_size":
            32,
        "epochs": 100, "augment": False}) # (c)
            head only
  train({"classes": B, "n_per_class": n,
        "init_from": "pre.pt"}) # (d) fine-tune
            all
  train({"classes": B, "n_per_class": n}) # (e)
       from scratch
```

One minute on a GPU, five on a CPU. Collect validation accuracy vs. n, four curves on one axis.

- **(a)** Train on five classes: the conv blocks now turn any image into 128 numbers.

E20 (start now, finish at home): what does a trained network know about classes it never saw?

```
from cnn_lab import train, nearest_mean
train({"classes": [0,1,2,3,4], "epochs": 10,
       "save_weights": "pre.pt"}) # (a) plane..deer
B = [5,6,7,8,9] # dog..truck
for n in (10, 100, 1000): # per class
  nearest_mean("pre.pt", B, n) # (b) no training
  train({"classes": B, "n_per_class": n, "
       init_from": "pre.pt",
         "freeze": True, "lr": 1e-2, "batch_size":
              32,
         "epochs": 100, "augment": False}) # (c)
              head only
  train({"classes": B, "n_per_class": n,
         "init_from": "pre.pt"}) # (d) fine-tune
              all
  train({"classes": B, "n_per_class": n}) # (e)
       from scratch
```

One minute on a GPU, five on a CPU. Collect validation accuracy vs. n, four curves on one axis.

- **(a)** Train on five classes: the conv blocks now turn any image into 128 numbers.
- **(b) No training.** The *other* five classes through the frozen blocks; each class gets the mean of its n feature vectors; a new image goes to the nearest mean. Not one gradient step on dogs or ships.

E20 (start now, finish at home): what does a trained network know about classes it never saw?

```
from cnn_lab import train, nearest_mean
train({"classes": [0,1,2,3,4], "epochs": 10,
       "save_weights": "pre.pt"}) # (a) plane..deer
B = [5,6,7,8,9] # dog..truck
for n in (10, 100, 1000): # per class
  nearest_mean("pre.pt", B, n) # (b) no training
  train({"classes": B, "n_per_class": n, "
       init_from": "pre.pt",
         "freeze": True, "lr": 1e-2, "batch_size":
              32,
         "epochs": 100, "augment": False}) # (c)
              head only
  train({"classes": B, "n_per_class": n,
         "init_from": "pre.pt"}) # (d) fine-tune
              all
  train({"classes": B, "n_per_class": n}) # (e)
       from scratch
```

One minute on a GPU, five on a CPU. Collect validation accuracy vs. n, four curves on one axis.

- **(a)** Train on five classes: the conv blocks now turn any image into 128 numbers.
- **(b) No training.** The *other* five classes through the frozen blocks; each class gets the mean of its n feature vectors; a new image goes to the nearest mean. Not one gradient step on dogs or ships.
- **(c)** Train only the head: 645 numbers; a linear head needs its own recipe. **(d)** Fine-tune everything. **(e)** From scratch: the baseline.

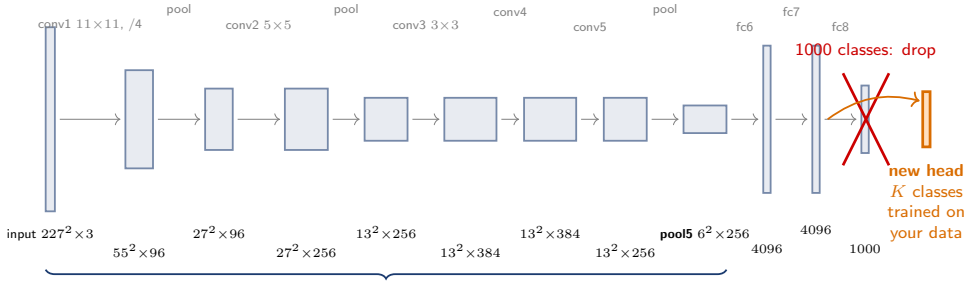
E20 (start now, finish at home): what does a trained network know about classes it never saw?

```
from cnn_lab import train, nearest_mean
train({"classes": [0,1,2,3,4], "epochs": 10,
       "save_weights": "pre.pt"}) # (a) plane..deer
B = [5,6,7,8,9] # dog..truck
for n in (10, 100, 1000): # per class
  nearest_mean("pre.pt", B, n) # (b) no training
  train({"classes": B, "n_per_class": n, "
       init_from": "pre.pt",
         "freeze": True, "lr": 1e-2, "batch_size":
              32,
         "epochs": 100, "augment": False}) # (c)
              head only
  train({"classes": B, "n_per_class": n,
         "init_from": "pre.pt"}) # (d) fine-tune
              all
  train({"classes": B, "n_per_class": n}) # (e)
       from scratch
```

One minute on a GPU, five on a CPU. Collect validation accuracy vs. n, four curves on one axis.

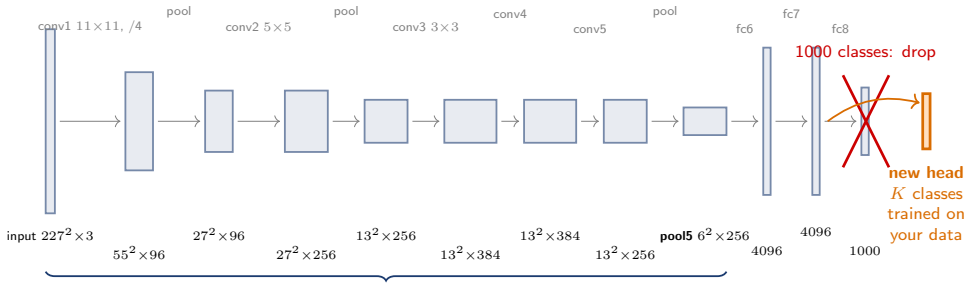
- **(a)** Train on five classes: the conv blocks now turn any image into 128 numbers.
- **(b) No training.** The *other* five classes through the frozen blocks; each class gets the mean of its n feature vectors; a new image goes to the nearest mean. Not one gradient step on dogs or ships.
- **(c)** Train only the head: 645 numbers; a linear head needs its own recipe. **(d)** Fine-tune everything. **(e)** From scratch: the baseline.
- **Predict the four curves vs. n before you run.** Where is (b) relative to (e)? Where do they cross?

Transfer learning: keep the backbone, replace the head



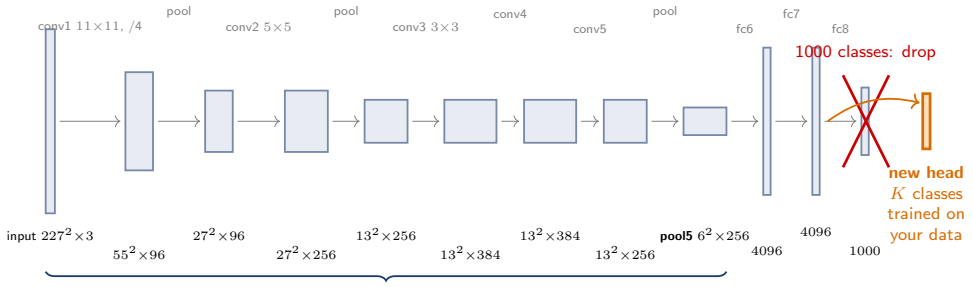
backbone: trained on ImageNet, 1.3M photos. Keep it.
frozen (features only) or fine-tuned (small η)

Transfer learning: keep the backbone, replace the head



- The stencils and the deep features are about images in general (the two AlexNet figures); only the last layer is about the 1000 ImageNet classes. Cut it off, add a linear layer with K outputs, train on your n images: the head only when n is small, everything with a small learning rate when n is larger.

Transfer learning: keep the backbone, replace the head



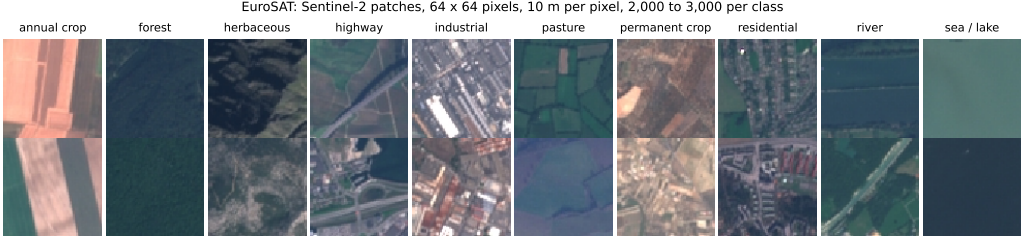
- The stencils and the deep features are about images in general (the two AlexNet figures); only the last layer is about the 1000 ImageNet classes. Cut it off, add a linear layer with K outputs, train on your n images: the head only when n is small, everything with a small learning rate when n is larger.
- The same move everywhere: a language model pretrained on the web, then a head or a light fine-tune for your task; speech, protein and molecule models alike. *Pretrain on what is plentiful, adapt on what is scarce.*

Borrow a trained network: transfer learning

A ResNet-18 trained on ImageNet (1.3M photos, 1000 classes) has learned edges, textures, parts. Keep the **backbone**, replace the **head**:

```
from torchvision.models import resnet18
m = resnet18(weights="IMAGENET1K_V1")
m.requires_grad_(False) # frozen: features only
m.fc = nn.Linear(512, K) # a new head, K classes
# fine-tune: no freeze; AdamW lr 1e-4, few
    epochs
```

EuroSAT (Helber et al. 2019): Sentinel-2 satellite patches, 64×64 pixels, 10 land-use classes, $27{,}000$ patches. Not ImageNet photos.



Borrow a trained network: transfer learning

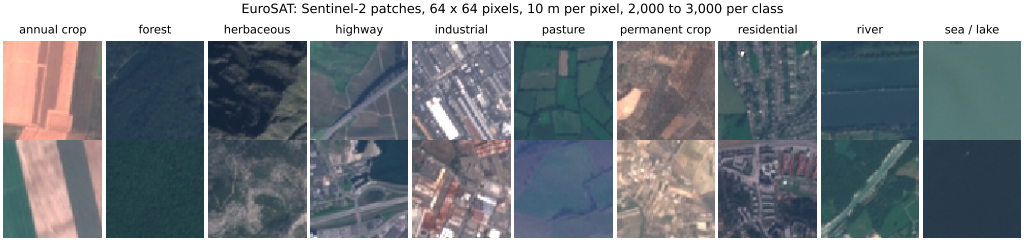
A ResNet-18 trained on ImageNet (1.3M photos, 1000 classes) has learned edges, textures, parts. Keep the **backbone**, replace the **head**:

```
from torchvision.models import resnet18
m = resnet18(weights="IMAGENET1K_V1")
m.requires_grad_(False) # frozen: features only
m.fc = nn.Linear(512, K) # a new head, K classes
# fine-tune: no freeze; AdamW lr 1e-4, few
    epochs
```

EuroSAT (Helber et al. 2019): Sentinel-2 satellite patches, 64×64 pixels, 10 land-use classes, $27{,}000$ patches. Not ImageNet photos.

Key point

With 10 images per class the borrowed features give 78%. For a small scientific image set, try this before training anything.



E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. **Challenge**: build the four models yourself and pass `model=`; for the frozen one, extract the features once. The sheet has both options.

E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. **Challenge**: build the four models yourself and pass `model=`; for the frozen one, extract the features once. The sheet has both options.

- **mlp**, **scratch**: the two ways we know. The baseline that must fail first at $n = 100$; and does a ResNet from scratch do better than an MLP with 10 images per class?

E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. **Challenge**: build the four models yourself and pass `model=`; for the frozen one, extract the features once. The sheet has both options.

- **mlp**, **scratch**: the two ways we know. The baseline that must fail first at $n = 100$; and does a ResNet from scratch do better than an MLP with 10 images per class?
- **frozen**: the backbone never saw a satellite image; $5{,}130$ numbers train. If this works, what ImageNet taught it was not about ImageNet.

E21 (homework): transfer learning on EuroSAT

```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. **Challenge**: build the four models yourself and pass `model=`; for the frozen one, extract the features once. The sheet has both options.

- **mlp**, **scratch**: the two ways we know. The baseline that must fail first at $n = 100$; and does a ResNet from scratch do better than an MLP with 10 images per class?
- **frozen**: the backbone never saw a satellite image; $5{,}130$ numbers train. If this works, what ImageNet taught it was not about ImageNet.
- **finetune**: everything, from the pretrained start, small learning rate. Expect it to overtake the frozen features as n grows.

E21 (homework): transfer learning on EuroSAT

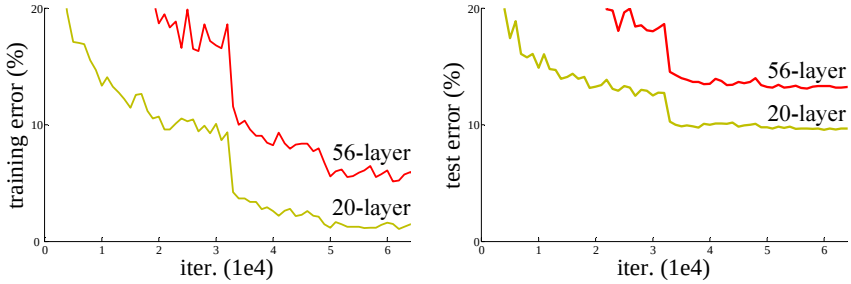
```
from eurosat_lab import train # same interface
for n in (100, 300, 1000): # labelled images
  train({"method": "mlp", "n": n})
  train({"method": "scratch", "n": n})
  train({"method": "frozen", "n": n})
  train({"method": "finetune", "n": n})
```

EuroSAT: $27{,}000$ Sentinel-2 patches, 64×64, 10 land-use classes; $2{,}000$ validation, $5{,}000$ test, n labelled for training. **Challenge**: build the four models yourself and pass `model=`; for the frozen one, extract the features once. The sheet has both options.

- **mlp**, **scratch**: the two ways we know. The baseline that must fail first at $n = 100$; and does a ResNet from scratch do better than an MLP with 10 images per class?
- **frozen**: the backbone never saw a satellite image; $5{,}130$ numbers train. If this works, what ImageNet taught it was not about ImageNet.
- **finetune**: everything, from the pretrained start, small learning rate. Expect it to overtake the frozen features as n grows.
- Predict the four curves vs. n before running. On a CPU: **frozen** and **mlp** at every n; **scratch** and **finetune** at $n \leq 300$, or with `size=64`.

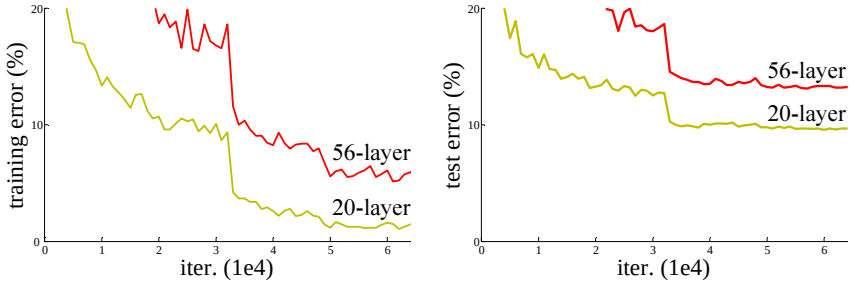
Going deep

Deeper is worse?



He, Zhang, Ren, Sun 2016, Fig. 1. CIFAR-10, plain networks of 3×3 convs with BatchNorm.

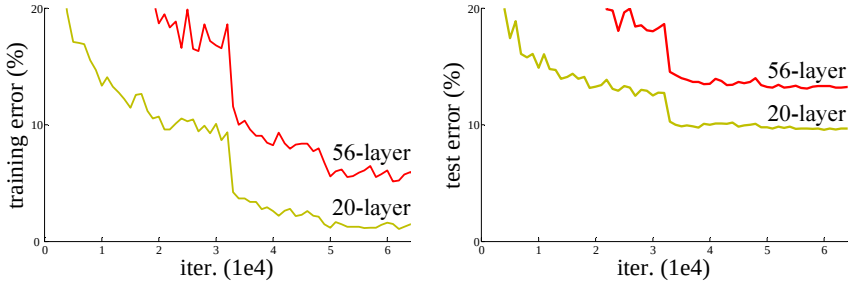
Deeper is worse?



He, Zhang, Ren, Sun 2016, Fig. 1. CIFAR-10, plain networks of 3×3 convs with BatchNorm.

- A 56-layer plain net against a 20-layer one: the deeper one has the higher *training* error. Not overfitting: the optimizer cannot use the extra layers. BatchNorm is on, so not the week-2 signal collapse either.

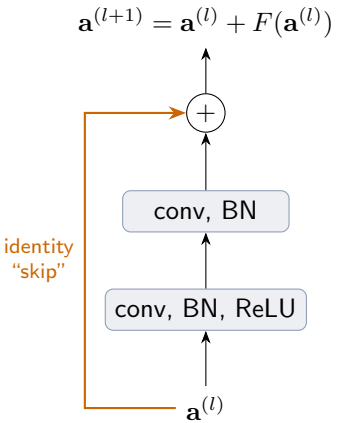
Deeper is worse?



He, Zhang, Ren, Sun 2016, Fig. 1. CIFAR-10, plain networks of 3×3 convs with BatchNorm.

- A 56-layer plain net against a 20-layer one: the deeper one has the higher *training* error. Not overfitting: the optimizer cannot use the extra layers. BatchNorm is on, so not the week-2 signal collapse either.
- The 56-layer net *contains* the 20-layer solution (set 36 layers to the identity); gradient descent does not find it. A deep stack of layers has no easy way to be the identity.

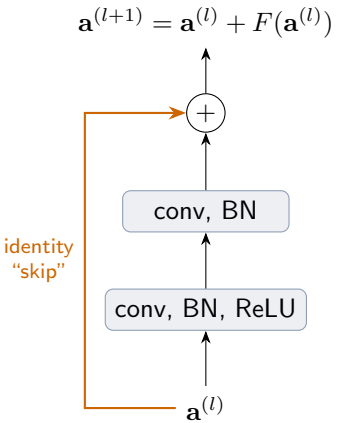
Going deep: skip connections



He, Zhang, Ren, Sun 2016 (ResNet).

- Last week: through depth, signal and gradient are a *product* of one factor per layer. Add the identity around every block and the product becomes a *sum*: the gradient reaches layer 1 through the skips, whatever the blocks do.

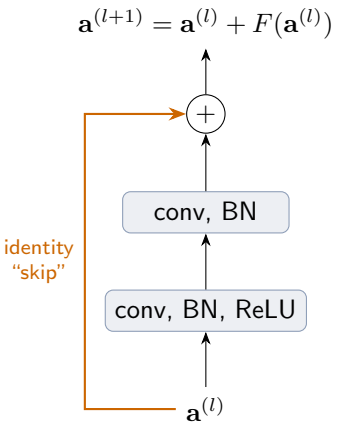
Going deep: skip connections



He, Zhang, Ren, Sun 2016 (ResNet).

- Last week: through depth, signal and gradient are a *product* of one factor per layer. Add the identity around every block and the product becomes a *sum*: the gradient reaches layer 1 through the skips, whatever the blocks do.
- The block learns a *residual* F; $F = 0$ is easy, so deeper is never worse in practice. 18, 50, 152 layers.

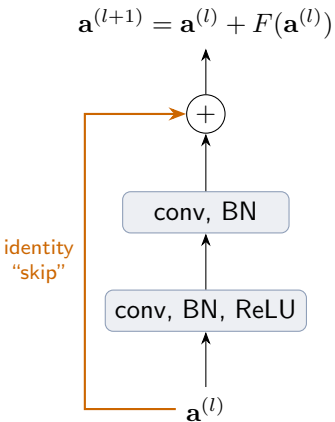
Going deep: skip connections



He, Zhang, Ren, Sun 2016 (ResNet).

- Last week: through depth, signal and gradient are a *product* of one factor per layer. Add the identity around every block and the product becomes a *sum*: the gradient reaches layer 1 through the skips, whatever the blocks do.
- The block learns a *residual* F; $F = 0$ is easy, so deeper is never worse in practice. 18, 50, 152 layers.
- **ResNet-18**: $[64, 128, 256, 512]$ channels, two blocks each, 11M parameters; 94–95% on CIFAR-10 from scratch, 70% top-1 on ImageNet. The standard backbone.

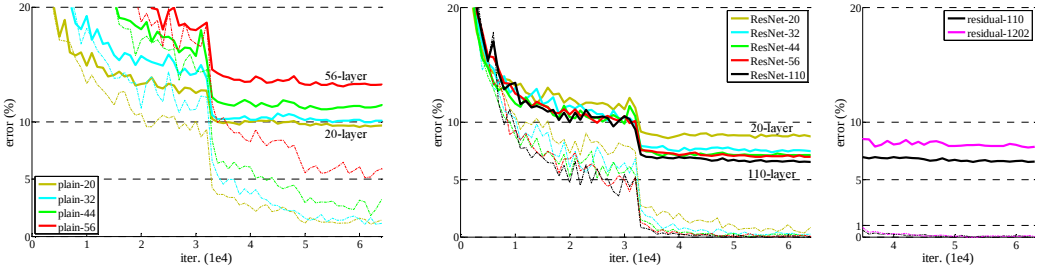
Going deep: skip connections



He, Zhang, Ren, Sun 2016 (ResNet).

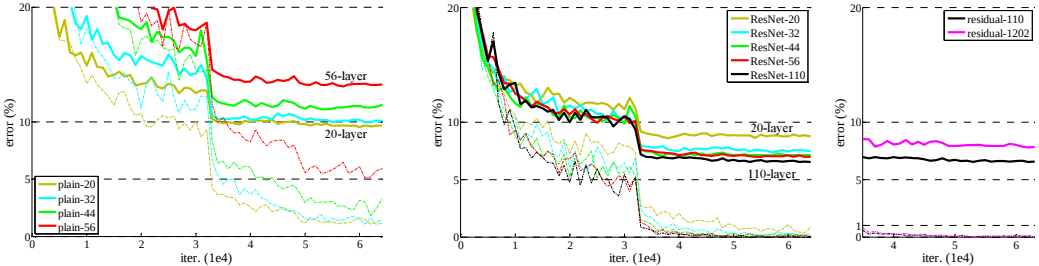
- Last week: through depth, signal and gradient are a *product* of one factor per layer. Add the identity around every block and the product becomes a *sum*: the gradient reaches layer 1 through the skips, whatever the blocks do.
- The block learns a *residual* F; $F = 0$ is easy, so deeper is never worse in practice. 18, 50, 152 layers.
- **ResNet-18**: $[64, 128, 256, 512]$ channels, two blocks each, 11M parameters; 94–95% on CIFAR-10 from scratch, 70% top-1 on ImageNet. The standard backbone.
- The same trick everywhere: transformers are residual stacks (week 6). Inside: BatchNorm, global average pooling, last week's recipe.

With skips, deeper is better



He et al. 2016, Fig. 6. CIFAR-10; dashed: training error, bold: test error. Left: plain nets. Middle: the same depths with skips. Right: 110 and 1202 layers.

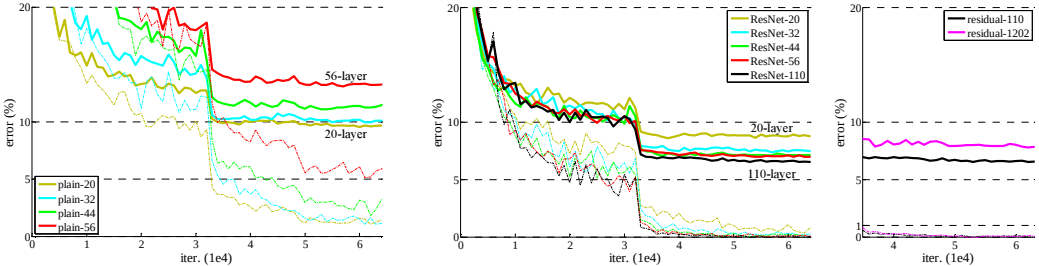
With skips, deeper is better



He et al. 2016, Fig. 6. CIFAR-10; dashed: training error, bold: test error. Left: plain nets. Middle: the same depths with skips. Right: 110 and 1202 layers.

- Plain: 20, 32, 44, 56 layers get worse in that order. With the identity around every pair of convs: better in that order, down to 6.4% at 110 layers.

With skips, deeper is better



He et al. 2016, Fig. 6. CIFAR-10; dashed: training error, bold: test error. Left: plain nets. Middle: the same depths with skips. Right: 110 and 1202 layers.

- Plain: 20, 32, 44, 56 layers get worse in that order. With the identity around every pair of convs: better in that order, down to 6.4% at 110 layers.
- At 1202 layers, 19M parameters, the training error is still 0 and the test error rises again: the generalization failure, back where it belongs. Skips fix the optimization, nothing else.

E22 (homework): depth with and without skips

```
train({"convs_per_block": 4, "batchnorm":
    False,
      "epochs": 10}) # 12 convs, plain
train({"convs_per_block": 4, "batchnorm":
    False,
      "epochs": 10, "skip": True}) # the same,
           with skips
```

$4\times$ the default cost: ≈ 20 s per epoch on a GPU; on a CPU 5 epochs. Collect the training loss vs. epoch of both.

E22 (homework): depth with and without skips

```
train({"convs_per_block": 4, "batchnorm":
    False,
      "epochs": 10}) # 12 convs, plain
train({"convs_per_block": 4, "batchnorm":
    False,
      "epochs": 10, "skip": True}) # the same,
           with skips
```

$4\times$ the default cost: ≈ 20 s per epoch on a GPU; on a CPU 5 epochs. Collect the training loss vs. epoch of both.

- `skip=True`: every conv gets the identity around it (a 1×1 projection where the channels change), $y = \mathrm{ReLU}(\mathrm{conv}(x) + x)$.

E22 (homework): depth with and without skips

```
train({"convs_per_block": 4, "batchnorm":
    False,
      "epochs": 10}) # 12 convs, plain
train({"convs_per_block": 4, "batchnorm":
    False,
      "epochs": 10, "skip": True}) # the same,
           with skips
```

$4\times$ the default cost: ≈ 20 s per epoch on a GPU; on a CPU 5 epochs. Collect the training loss vs. epoch of both.

- `skip=True`: every conv gets the identity around it (a 1×1 projection where the channels change), $y = \mathrm{ReLU}(\mathrm{conv}(x) + x)$.
- The same twelve layers, the same parameters up to the 1×1 projections, the same recipe. Only the function class changes: each conv now learns a correction to its input.

E22 (homework): depth with and without skips

```
train({"convs_per_block": 4, "batchnorm":
    False,
      "epochs": 10}) # 12 convs, plain
train({"convs_per_block": 4, "batchnorm":
    False,
      "epochs": 10, "skip": True}) # the same,
           with skips
```

$4\times$ the default cost: ≈ 20 s per epoch on a GPU; on a CPU 5 epochs. Collect the training loss vs. epoch of both.

- `skip=True`: every conv gets the identity around it (a 1×1 projection where the channels change), $y = \mathrm{ReLU}(\mathrm{conv}(x) + x)$.
- The same twelve layers, the same parameters up to the 1×1 projections, the same recipe. Only the function class changes: each conv now learns a correction to its input.
- Predict both training curves first: how much faster, how much lower? Then, with a GPU, BatchNorm back on, both again: how much of the gap does BatchNorm close, and what is left for the skips?

Part I, the map

failure	what we measured	what fixed it
optimization	$\ln 10$ forever; crawl, oscillate, diverge; the floor	normalization, momentum/Adam, schedules, init, BatchNorm (week 2)
generalization	training 100%, validation 55	priors: augmentation, decay, early stopping, dropout (week 2)
approximation	$\approx 60\%$ for the MLP, whatever we did	the ansatz: convolution, pooling, skips; a borrowed backbone (week 3)

Part I, the map

failure	what we measured	what fixed it
optimization	$\ln 10$ forever; crawl, oscillate, diverge; the floor	normalization, momentum/Adam, schedules, init, BatchNorm (week 2)
generalization	training 100%, validation 55	priors: augmentation, decay, early stopping, dropout (week 2)
approximation	$\approx 60\%$ for the MLP, whatever we did	the ansatz: convolution, pooling, skips; a borrowed backbone (week 3)

- **Homework**: E19 (take things away), E20 (what a trained network knows about classes it never saw), E21 (transfer learning on EuroSAT), E22 (depth with and without skips). Report as a GitHub issue.

Part I, the map

failure	what we measured	what fixed it
optimization	$\ln 10$ forever; crawl, oscillate, diverge; the floor	normalization, momentum/Adam, schedules, init, BatchNorm (week 2)
generalization	training 100%, validation 55	priors: augmentation, decay, early stopping, dropout (week 2)
approximation	$\approx 60\%$ for the MLP, whatever we did	the ansatz: convolution, pooling, skips; a borrowed backbone (week 3)

- **Homework**: E19 (take things away), E20 (what a trained network knows about classes it never saw), E21 (transfer learning on EuroSAT), E22 (depth with and without skips). Report as a GitHub issue.
- **Quiz in class on Monday 2 Nov.** Easy if you did the homework and understood it.

Part I, the map

failure	what we measured	what fixed it
optimization	$\ln 10$ forever; crawl, oscillate, diverge; the floor	normalization, momentum/Adam, schedules, init, BatchNorm (week 2)
generalization	training 100%, validation 55	priors: augmentation, decay, early stopping, dropout (week 2)
approximation	$\approx 60\%$ for the MLP, whatever we did	the ansatz: convolution, pooling, skips; a borrowed backbone (week 3)

- **Homework**: E19 (take things away), E20 (what a trained network knows about classes it never saw), E21 (transfer learning on EuroSAT), E22 (depth with and without skips). Report as a GitHub issue.
- **Quiz in class on Monday 2 Nov.** Easy if you did the homework and understood it.
- Next week: Mon: Me, autoencoders. Thu: Zohar, VAEs and anomaly detection.